GenAI HandsOn 3

Name: Arnav Sinha
SRN: PES2UG23CS092
Sec: B

Notebook 1: Advanced RAG & Retrieval

This notebook focuses on improving retrieval quality by addressing the limitations of Naïve RAG. Traditional RAG systems fail due to vocabulary mismatch (user vs document language) and top-K noise (irrelevant retrieved documents).

To solve this, two retrieval methods are introduced:

- BM25 (Sparse Retrieval):
  Works on keyword matching using TF-IDF principles. It is highly effective for exact matches, technical terms, and proper nouns.
- SBERT (Dense Retrieval):
  Converts text into embeddings and retrieves documents based on semantic similarity. It performs well when queries and documents use different wording but similar meaning.

Since both methods have strengths and weaknesses, they are combined using Hybrid Retrieval. The fusion technique used is Reciprocal Rank Fusion (RRF), which merges rankings from both retrievers to produce a balanced result.

Additionally, ColBERT introduces token-level matching using late interaction (MaxSim), improving fine-grained relevance between query and document tokens.

Tools Used

- rank-bm25 → BM25 implementation
- sentence-transformers → SBERT embeddings
- numpy → vector operations

Notebook 2: Re-Ranking & Query Expansion

This notebook enhances retrieval further by improving both query understanding and document ranking.

First, Query Expansion is introduced to solve vague or short queries:

- HyDE: Uses an LLM (Gemini) to generate a hypothetical answer, which is then used as a better search query.

- Multi-Query Retrieval: Generates multiple paraphrased queries and retrieves results for all, increasing coverage.

Next, Cross-Encoder Re-Ranking is applied:

- Unlike SBERT (bi-encoder), it evaluates query-document pairs together.
- Produces highly accurate relevance scores.
- Used as a second-stage filter after retrieval.

Finally, the notebook integrates everything into a full pipeline:

Query Expansion → Hybrid Retrieval → Re-Ranking → LLM Generation

This pipeline significantly improves accuracy by:

- Expanding queries (better recall)
- Combining retrievers (better retrieval)
- Re-ranking results (better precision)

Tools Used

- sentence-transformers → Cross-Encoder model
- langchain-google-genai → Gemini (HyDE, query generation)
- langchain-community → MultiQueryRetriever
- FAISS → vector storage

Notebook 3: Generation Enhancement

This notebook focuses on improving the generation side of RAG systems.

First, it explains when to use:

- Prompting
- RAG
- Fine-tuning
- Pre-training

This helps choose the right strategy based on problem complexity.

Next, LoRA (Low-Rank Adaptation) is introduced:

- Instead of training full models, only small matrices are trained.

- Reduces memory and computation cost.
- Enables efficient fine-tuning using PEFT (Parameter-Efficient Fine-Tuning).

Then, Quantization is covered:

- Converts model weights to lower precision (FP16, INT8).
- Reduces memory usage and speeds up inference.
- Trade-off: slight accuracy loss.

Finally, Mixture of Experts (MoE) is explained:

- Uses multiple specialized models (experts).
- A router selects which expert to use.
- Improves scalability and efficiency.

Tools Used

- transformers → model loading
- peft → LoRA implementation
- langchain-groq → MoE-style routing with LLMs
- numpy → quantization math

<u>Assignment Notebook</u>

This notebook implements a complete Advanced RAG system by combining all techniques learned.

First, a document corpus is created with multiple AI/ML topics to simulate a real knowledge base.

Then, a HybridRetriever is built:

- Uses BM25 + SBERT
- Combines results using RRF
- Tracks individual ranks for analysis

Next, Query Expansion (HyDE or Multi-query) is applied to improve retrieval for vague queries.

A Cross-Encoder Re-Ranker is then used to refine results by scoring query-document pairs.

All components are integrated into a final function: advanced_rag(query)

Finally, the system is compared with Naïve RAG to demonstrate improvements in:

- Relevance
- Accuracy
- Context understanding

Tools Used

- rank-bm25 → keyword retrieval
- sentence-transformers → embeddings + re-ranking
- Gemini API → query expansion & generation
- FAISS → vector search